Verifiable Generative Views: Rendering, Editing, and Auditing LLM-Generated Interfaces over Bound Data

Rawi: the view-rendering layer of DeeWaanAI OS

Youssef Hariri
DeeWaanAI
github.com/DeeWaanAI

October 2026

Abstract

Language models can already generate interfaces on demand. What they generate is unaccountable: numbers on screen are copies, edits go nowhere durable, and the evidence behind a claim is a promise, not a proof. We present **Rawi**, the view-rendering layer of DeeWaanAI OS, built on one rule: a view node carries a *binding reference*, never a value. Rendered over Qafiya, a provenance-first data substrate, one artifact becomes three things at once: the proof source (every value is recomputed from its binding at render time), the edit target (a cell edit is an *edit binding* with atomic write-back and transitive invalidation), and the audit pointer (every claim exposes its binding, source rows, and last check result through one affordance). A systematic review of 797 records (42 synthesised) found no evidence in the surveyed set of prior work occupying this seam. We evaluate on a frozen corpus under a self-verifying analysis spine. A scripted end-to-end edit closes the durability claim left open by the substrate paper; 100 random edits through views re-prove at 100%; and in an offline estimate against a full-regeneration baseline (no conversation state, no caching, no incremental regeneration), rendering spec-once costs 135× fewer model tokens than per-interaction regeneration. We report what the system provably cannot do: when data and claim share the same error, agreement looks like verification, and the naive resolver crosses the interactive threshold between 100 and 1,000 nodes. Rawi is named for the classical Arabic transmitter of poetry, who recited what he did not compose — and produced his chain of transmission (*isnād*) on demand.

1 Introduction

Ask a current AI assistant to "show Q3 expenses" and it draws something convincing. Ask where the total came from and the answer is a paragraph. Change a number and the paragraph is stale but does not know it. The interface is generated; the accountability is not.

This is not a model-quality problem. It is an architecture problem. The generated interface is a *copy* of the data, produced once, with no durable connection back to its source. Every downstream failure — unverifiable claims, lost edits, silent staleness — follows from that single fact.

The companion substrate paper (Qafiya) established a data model in which every value or claim is joined to its source by a *binding*: a first-class object that proves the value equals its data, serves as the target of durable edits, and carries a verifiable claim. That paper proved the substrate and left one claim open: durability *end-to-end*, through a real user-facing edit, with the invalidation visible to the user. This paper closes that claim and introduces the layer where "re-run it yourself" becomes visible: the renderer.

Contributions.

1. **A verifiable view grammar** (`dvr/1.0`): four view kinds, four node roles, one rule — data-bearing nodes carry `binding_ref`, never values (§4).
2. **Three mechanics on one artifact**: render = proof source, edit = binding, audit = recomputable check, including the ▶*evidence* affordance (§4.3).
3. **Frozen, spine-verified experiments**: end-to-end durability (closing the substrate's open claim), 100% round-trip re-proving over 100 edits, and a 135× token-cost advantage over per-interaction regeneration (§6).
4. **Measured limits**: shared-error blindness, a located scale ceiling, last-writer-wins concurrency, and the grammar's expressiveness floor (§7).

Provenance of this paper. A systematic literature review (797 records identified, 749 deduplicated, 110 screened, 42 synthesised; OpenAlex and arXiv queried programmatically on 2026-10-19, all citations verified against registry metadata) grounds §3. All experimental inputs were frozen with SHA-256 hashes before any run; every headline number is recomputed from raw artifacts by a spine script (`spine.py`, exit 0 required). Code: `github.com/DeeWaanAI` (AGPL-3.0).

2 Background: the Qafiya substrate

Rawi consumes, and does not modify, Qafiya spec v1.1. Three substrate facts matter here.

The binding. One object, three jobs: render source, edit target, audit pointer. A binding records its sources (dataset, filter, field, verbatim excerpt), a deterministic *derivation* over a closed function set (`sum, avg, count, min, max`), a cached value, the claim tuple (statistic, device, authoritative artifact), and a check result.

The two-layer check. Layer 1 is a deterministic proof: the value is recomputed from current data; quarantined artifacts fail by construction; gaps are hard stops. Layer 2 is an advisory semantic flag: direction and comparison claims are recomputed from *both sides* and mismatches are flagged with the recomputed value shown. Mechanism/causal claims are never auto-passed.

Durability. Edits are atomic (write-temp-then-rename), prior states are retained in an append-only history, and changing a record transitively invalidates every dependent binding to PENDING. The substrate paper demonstrated this at the data layer and explicitly left the user-visible, end-to-end loop open.

3 Related work

Five literatures each hold one piece of a verifiable generative view. None holds the seam between them.

3.1 Declarative view grammars

A small composable spec can render rich interactive views: Vega-Lite [1], its Python binding Altair [2], tree grammars (GoTree [3]), responsive grammars (Cicero [4]), procedural generation grammars (Atlas [5]), and annotation grammars (AnnoGram [6]). Pre-LLM natural-language interfaces compiled intent into these grammars (Data2Vis [7], NL4DV [8]). All of them assume the spec embeds or points at finished, trusted data. The spec is a rendering recipe, not a provenance

object: nothing in a Vega-Lite node knows which source rows produced it, and nothing downstream can re-verify it. Rawi inherits this lineage's compositional style and changes what a node carries.

3.2 LLM-generated views and generative UI

LLMs moved authoring from "write the spec" to "say what you want": Data Formulator [9] and its successor [10], grammar-agnostic generation (LIDA [11]), LLM-authored data articles (DataTales [12]), with benchmarks now measuring generation quality (VisEval [13], and evaluations of LLM visualization generation [14]). Generative UI is now a named subfield: generative interfaces for language models [15], a working definition [16], personalization [17], DSL-brick approaches (LEGOUI [18]), software-as-content dynamic applications [19], just-in-time interfaces from intent (Bespoke [20]), and conversational programming assistance [21]. In every surveyed system the artifact's tie to data is ephemeral: baked in at generation time, or fetched by generated code with no durable binding. Editing means re-prompting; auditing means trusting. None reports the cost structure of per-interaction regeneration — the gap our E3 experiment fills.

3.3 Editing through views

Direct manipulation of bound visualizations is well studied: Data Illustrator [22], Lyra 2 [23], reusable graphical structures (StructGraphics [24]), direct manipulation of code-based visualizations (DirectVis [25]), and natural-language editing of visual knowledge graphs [26]. Idyll [27] established reactive markup for interactive articles. These systems make views editable, but the edit lands in the visualization or in opaque runtime state — not in a first-class provenance object naming the exact record and field. And none answers the follow-up: after the edit, which claims derived from that record are now stale? Write-back exists; accountable write-back with transitive invalidation does not.

3.4 Evidence and audit affordances

Interfaces that surface evidence are a growing literature: provenance in LLM-assisted writing (HaLLMark [28]), citation and attribution scoring (ALCE [29], FActScore [30]) — post-hoc metrics, not live affordances. Uncertainty is under-communicated even by experts [31]; provenance-density displays mitigate a measured "transparency penalty" [32]; explanation interfaces risk an illusion of explanatory depth [34] and motivate graspable rather than merely explainable AI [33]; analytic-pipeline explanations (XNLI [35]) and analytic-provenance instrumentation [36] complete the space. The field converged on *showing* provenance and *scoring* faithfulness — separately. No surveyed interface exposes a recomputable check behind the affordance: an evidence panel whose check can fail, live, in front of the user.

3.5 Reactive recompute and invalidation

Keeping derived views correct after change is solved machinery in three disconnected communities: reactive notebooks — whose silent failure modes are documented [37] — live-programming GUIs (Glinda [38]), distributed reactive languages (Meerkat [39]), grounded agentic analysis in reactive notebooks (marimo-lens [40]), and incremental view maintenance in databases [41, 42]. Invalidation is a solved engineering problem — and a solved *invisible* one. Rawi's contribution here is making staleness a visible, checked state in a generated UI, wired through the provenance substrate.

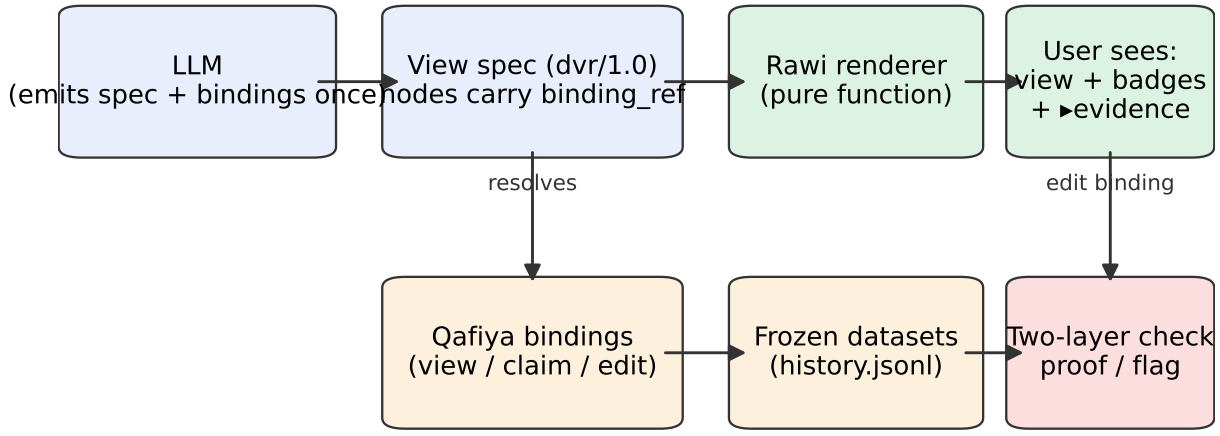


Figure 1: Architecture. The model pays once (spec + bindings). The renderer is a pure function of (spec, bindings, datasets). Edits flow down as edit bindings; check states flow up as visible badges.

3.6 The gap

Grammars render but embed values; generative UI binds ephemerally; direct-manipulation edits are not provenance objects; audit affordances are not recomputable; reactive machinery is invisible. The seam is empty, and it is where Rawi sits.

4 Design

4.1 The grammar (`dvr/1.0`)

A view is a declarative JSON document. Four kinds — `table, list, tree, deck` — and four node roles — `value, label, claim, structure` — form closed sets; new entries require a spec version bump, the same discipline as the substrate's closed derivation set.

```
{"spec":"dvr/1.0", "id":"v_exp", "kind":"table",
"nodes":[{"id":"cell:total_q3", "role":"value",
          "binding_ref":"b_tot_q3", "editable":false},
          {"id":"claim:trend", "role":"claim", "binding_ref":"b_trend"}]}
```

The validator enforces well-formedness by construction: unknown keys, kinds, or roles are rejected; `value` and `claim` nodes must reference exactly one existing binding; `label`/`structure` nodes must not; claim nodes are never editable; and fields that would embed data (`value`, `data`, `number`) are forbidden on nodes outright. A view that carries a number does not parse.

4.2 Invariants

Eight testable properties, each mapped to an automated test:

1. No rendered number exists in the view spec (validator-enforced).

2. Every rendered value traces to exactly one binding (resolution-enforced).
3. Byte-identical re-render under unchanged inputs (purity).
4. Every edit leaves an edit binding and a history entry.
5. After any edit, every transitively dependent binding is PENDING before it is anything else.
6. Every claim node renders ▶evidence; its verbatim rows byte-match the dataset.
7. Coverage is rendered with every view.
8. An unresolved binding is visibly unresolved — never silent, never guessed.

4.3 Three mechanics

Render = proof source. Each displayed value is recomputed from the binding's derivation over current data via the substrate's evaluator; a cached value is never printed without recompute. Failed resolution renders as UNRESOLVED. Every node carries a check badge; every view reports coverage, so a 33%-bound view looks different from a 100%-bound one.

Edit = binding. A cell edit emits an edit binding (`writeback: {dataset, record_id, field, op, value}`) that the substrate applies atomically, logs with the prior value, and uses to invalidate every transitively dependent binding to PENDING. The renderer then re-resolves and re-renders badges. The claim's assertion text is never altered by the renderer — that is what the semantic layer flags.

Audit = ▶evidence. One click on any claim reveals, resolved live: the binding (id, derivation, claim tuple), the verbatim source rows matching its filter, and the last two-layer check result with detail. The affordance is one click deep with a one-line summary by design, against the illusion-of-depth [34] and transparency-penalty [32] findings.

4.3.1 Focused evidence: showing the lines that matter

A first version of the panel showed the binding and *all* matching source rows. Early pilot use by the author exposed the flaw: the user saw data, but not *where the error is* nor *what the correct value is*. More evidence is not better evidence. The fix is a *focus block* computed at click time, with three elements in a fixed order:

1. **Claim versus data.** The assertion text and its expected value, next to the correct value recomputed from current data. For comparisons, both sides are re-evaluated and shown — the reader never takes either side on faith.
2. **The change trail.** Which records moved, from the append-only history: record id, field, prior → new. This is "where the error is," derived from the substrate's write log rather than guessed from the view.
3. **Deciding rows only.** The rows matching each side's filter — the evidence that actually decides the claim — not the whole table.

Worked example (the E1 scenario). After the edit of `r_051`, the flagged trend claim's panel reads:

```
claim says:  Q3 spend changed +0.4% vs Q2 (expected +0.4%)
data says:  +5.2%   ← the correct value
Q2 total = 10308.84  (21 rows)
Q3 total = 10846.71  (21 rows)
changed:  record r_051 · amount 820.89 → 1320.89
```

The logic is deliberately asymmetric. The claim's text is treated as untrusted prose and is never edited; the "correct value" is not the model's correction but the same derivation recomputed over current data — the check the substrate already ran, surfaced in human terms. The change trail exists because the substrate logs every edit; the panel is a read view over the audit log, not a second explanation channel that could drift from it. This is the design answer to the illusion-of-depth risk: the affordance shows the *decisive* evidence, in the order a skeptical reader asks for it, and stops.

5 Implementation

The prototype (`view-renderer/`, Python, ∼300 LOC) implements the validator (`specval.py`), resolver (`resolve.py`), pure HTML renderer (`render.py`), evidence assembler (`affordance.py`), and edit path (`edits.py`). It consumes Qafiya only through the `qafiya-core` public API (`apply_edit`, `invalidate`, `refresh_values`, `provenance_gate`, `semantic_sweep`); a static audit test enforces that no renderer module writes dataset files or references the substrate's internals. Ten functional tests cover grammar rejection (seven malformed-spec fixtures), resolution correctness under data tampering, render purity, atomic edits with history, invalidation visibility (PASS → PENDING → FLAG), evidence fidelity, coverage display, and contract compliance.

6 Evaluation

Discipline. Inputs frozen with SHA-256 hashes before any run (`corpus/manifest.json`); every headline number recomputed from raw artifacts by `spine.py` (exit 0 required); no number outside the spine's output appears in this paper. The corpus: 63 expense records across three quarters, six bindings (three period totals, one editable cell, two claims), one view spec, one pre-scripted 100-edit sequence.

6.1 E1 — durability end-to-end

Before. The substrate paper demonstrated invalidation at the data layer and left the user-visible end-to-end loop as an open claim.

Run. A scripted user edit raises record `r_051` from 820.89 to 1,320.89 through the editable cell of the rendered view (Figure 2).

After. The file updates atomically; history retains the prior value; four dependent bindings go PENDING, visibly. On re-check, totals re-prove PASS, and the trend claim — its text untouched — flips to FLAG: *"recompute shows +5.2%, claim says +0.4% (Q2=10308.84, Q3=10846.71)."* The claim was not silently repaired; it stands flagged, evidence one click away. The substrate's open claim is closed. (Trace: `results/raw/e1_trace.jsonl`.)

6.2 E2 — round-trip correctness

Before. Purity (invariant 3) and edit integrity (invariants 4–5) are untested under sustained mutation.

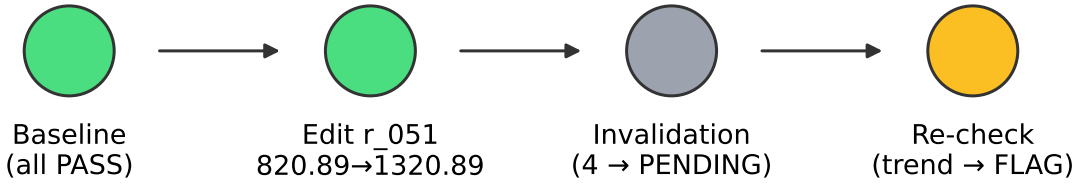


Figure 2: E1: the durability chain. The stale state is a visible, rendered state — not an implicit runtime assumption.

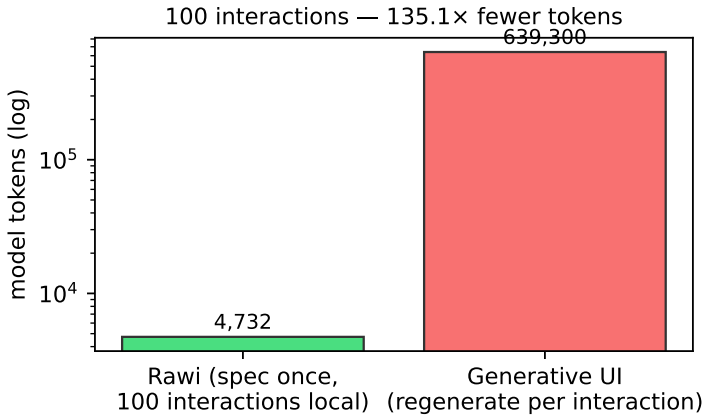


Figure 3: E3: model tokens for 100 interactions, log scale.

Run. The 100 frozen scripted edits are applied through the view; after each, the substrate gate must re-prove every binding and the file value must equal the scripted value.

After. 100/100 edits re-prove at 100% (1.0 s wall time). Zero gate failures, zero value mismatches. (`results/raw/e2_edits.jsonl`.)

6.3 E3 — rendering cost

Before. No surveyed generative-UI work reports the token cost of per-interaction regeneration.

Method (disclosed). Offline deterministic count with the actual Qwen2.5 tokenizer — the same instrument as the substrate's efficiency experiment, so the two papers' numbers are comparable. Arm DVR: one emission (dataset in, spec + bindings out) = 4,732 tokens; all 100 interactions local. Arm REGEN: per interaction, dataset in + full UI out = 6,393 tokens, with UI output estimated by a faithful HTML stand-in — disclosed as an estimate; no live API calls.

After. 100 interactions: 4,732 tokens (DVR) versus 639,300 (REGEN) — 135× cheaper, breakeven at 0.74 interactions (Figure 3). Spec-once wins from the first edit.

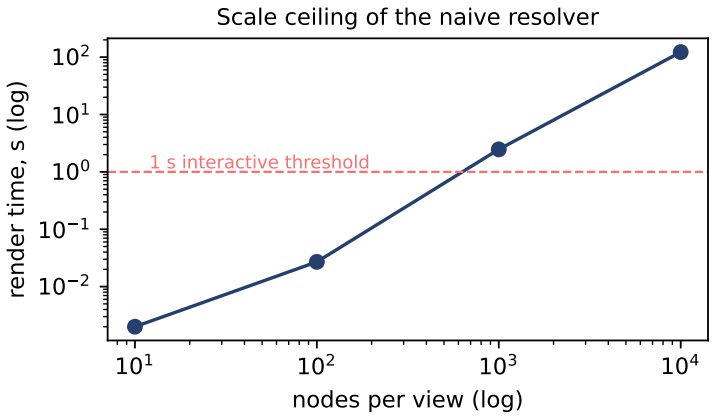


Figure 4: Scale ceiling of the naive resolver (log–log).

7 Limits

Measured, not hedged; each probe is a result whether it passes or fails (`tests/limits_results.json`).

Shared-error blindness. When data and claim share the same error, agreement passes every check. We constructed an internally consistent fabrication (a Q2 figure chosen so the false −12% claim verifies exactly): the system does not and cannot catch it. The remedy is an upstream second anchor — cost, not software. This is the substrate's provable limit, inherited and re-confirmed at the view layer.

Scale. The naive resolver re-evaluates every node's derivation over the full dataset at render time. The measured curve grows roughly quadratically in nodes over this corpus: 0.03 s at 100 nodes, 2.5 s at 1,000, 123 s at 10,000 (Figure 4) — crossing the 1-second interactive threshold between 100 and 1,000 nodes. A memoized derivation cache is the known fix; we publish the number, not the excuse.

Concurrency. Two simultaneous edits to one record: last-writer-wins, both entries preserved in history, no torn writes under atomic rename. No locking or multi-user semantics are claimed.

Expressiveness. Four kinds and four roles are deliberate minima. A frozen probe (L6, `results/raw/l6.json`) attempted the six shell-relevant view requests — prompt+answer, data table, claim with evidence, action confirmation, memory inspection, connection manager — and validated each attempted spec with the real grammar: 4 of 6 (66.7%) are expressible in `dvr/1.0`. The two gaps are named, not smoothed over: *action confirmation* needs action elements (approve/reject emitting effects, not data edits — Layer 3 territory), and a *connection manager* needs live status controls bound to service state rather than dataset bindings. Both are out of scope by design (spec §8); requests beyond the minima wait for a versioned extension.

Judgment. Layer-2 semantic checks are advisory flags; mechanism and causal claims are never auto-passed. Verification certifies coherence between claim and artifacts, never truth about the world.

8 Threats to validity

The review is single-pass over two databases with keyword-score screening plus manual curation, so recall is bounded by the query strings; the write-back and reactive blocks carried heavy domain noise

resolved at curation. The 42-work synthesis is corroborated by the substrate review's independent finding from the data-model side. Experiments run on one synthetic corpus of 63 records; E3's REGEN output is an estimated stand-in, not a live API measurement; no user study has been run. Several cited 2025–2026 works are preprints. These bound the claims, and the claims are scoped to match.

9 Discussion and future work

Rawi is Layer 2 of six. Above it, the action layer (Layer 3) will consume the same binding discipline for effects that go beyond data edits; the memory layer (Layer 4) will persist view state across sessions; the model router (Layer 5) will decide who emits specs. Below it, Qafiya's contract held without modification — the strongest available evidence that the binding primitive generalizes from substrate to rendering. The known engineering next steps are the memoized resolver, browser-native editing (currently the edit path is programmatic), and exercising the `tree` and `deck` kinds, which the grammar admits but the experiments have not stressed. A usability evaluation of the evidence affordance is deferred to field feedback after deployment.

10 Conclusion

The pieces of accountable generative interfaces exist in five separate literatures. Rawi joins them with one rule — a view node carries a binding reference, never a value — and the join holds under test: durability closed end-to-end, 100% round-trip re-proving, a 135× cost advantage in the offline full-regeneration comparison, and its blind spots measured and named. The classical *rāwī* recited poetry he did not compose and produced his chain of transmission on demand. That is the contract this layer offers the operating system above it.

AI-usage disclosure

This layer was built with an AI coding assistant (Kimi, Moonshot AI) operating under the author's direction and the project's frozen-artifact discipline. Division of labour: the assistant executed — literature search and screening code, prototype implementation, test and experiment scripts, spine, and draft text; the author decided — layer opening, design brief approval, scope (including the removal of the E4 user study), and all publication judgments. No LLM made unsupervised decisions.

Oversight episodes, logged as they occurred (claim map and decision log are the enforcement artifacts): (1) a fabricated-size claim ("∼600 LOC" vs. a measured 281 lines) caught by the data-map audit and corrected; (2) an unpersisted timing figure ("1.6 s") caught by the semantic sweep — the number existed only in console output; it was frozen into `e2_meta.json` and the draft corrected; (3) stale scale timings quoted from an earlier probe run, caught when the spine recomputed the curve from the frozen artifact. All three were corrected in the draft and are preserved in `paper/claim_map/semantic-claims.json` as the audit trail. The method itself — two-layer claim-to-evidence verification — is the authors' prior work, applied here to the assistant's own output.

References

[1] Satyanarayan, A., Moritz, D., Wongsuphasawat, K., Heer, J. (2016). Vega-Lite: A Grammar of Interactive Graphics. *IEEE TVCG*. doi:10.1109/tvcg.2016.2599030

[2] VanderPlas, J., Granger, B., Heer, J., et al. (2018). Altair: Interactive Statistical Visualizations for Python. *JOSS*. doi:10.21105/joss.01057

[3] Li, G., Tian, M., Xu, Q., et al. (2020). GoTree: A Grammar of Tree Visualizations. *ACM CHI*. doi:10.1145/3313831.3376297

[4] Kim, H., Rossi, R. A., Du, F., et al. (2022). Cicero: A Declarative Grammar for Responsive Visualization. *ACM CHI*. doi:10.1145/3491102.3517455

[5] Liu, Z., Chen, C., Morales, F., et al. (2021). Atlas: Grammar-based Procedural Generation of Data Visualizations. *IEEE VIS*. doi:10.1109/vis49827.2021.9623315

[6] Rahman, M. D., Rahat-Uz-Zaman, M., McNutt, A., et al. (2025). AnnoGram: An Annotative Grammar of Graphics Extension. *IEEE VIS*. doi:10.1109/vis60296.2025.00053

[7] Dibia, V. C., Demiralp, Ç. (2019). Data2Vis: Automatic Generation of Data Visualizations Using Seq2Seq RNNs. *IEEE CG&A*. doi:10.1109/mcg.2019.2924636

[8] Narechania, A. A., Srinivasan, A., Stasko, J. (2020). NL4DV: A Toolkit for Generating Analytic Specifications from Natural Language Queries. *IEEE TVCG*. doi:10.1109/tvcg.2020.3030378

[9] Wang, C., Thompson, J., Lee, B. (2023). Data Formulator: AI-powered Concept-driven Visualization Authoring. *IEEE TVCG*. doi:10.1109/tvcg.2023.3326585

[10] Wang, C., Lee, B., Drucker, S., et al. (2024). Data Formulator 2: Iterative Creation of Data Visualizations, with AI Transforming Data Along the Way. *arXiv:2408.16119*.

[11] Dibia, V. C. (2023). LIDA: Automatic Generation of Grammar-Agnostic Visualizations and Infographics using LLMs. *ACL*. doi:10.18653/v1/2023.acl-demo.11

[12] Sultanum, N., Srinivasan, A. (2023). DATATALES: Investigating the Use of LLMs for Authoring Data-Driven Articles. *IEEE VIS*. doi:10.1109/vis54172.2023.00055

[13] Chen, N., Zhang, Y., Xu, J., et al. (2024). VisEval: A Benchmark for Data Visualization in the Era of LLMs. *IEEE TVCG*. doi:10.1109/tvcg.2024.3456320

[14] Wang, X., Liang, C., Zheng, S., et al. (2024). Visualization Generation with Large Language Models: An Evaluation. *arXiv:2401.11255*.

[15] Chen, J., Zhang, Y., Zhang, Y., et al. (2025). Generative Interfaces for Language Models. *arXiv:2508.19227*.

[16] Lee, K. (2025). Towards a Working Definition of Designing Generative User Interfaces. *ACM DIS*. doi:10.1145/3715668.3736365

[17] Peng, Y.-H., Bigham, J. P., Wu, J. (2026). Efficient Personalization of Generative User Interfaces. *arXiv:2604.09876*.

[18] Zhou, Y., Yuan, M., Xu, H., et al. (2026). LEGOUI: Designing with UI-DSL Bricks to Balance Transparency and Controllability. *arXiv:2608.04293*.

[19] Xie, M., Xie, Y. (2026). Software as Content: Dynamic Applications as the Human-Agent Interaction Layer. *arXiv:2603.21334*.

[20] Nandy, P., Adalgeirsson, S. Ö., Sinha, A. K., et al. (2024). Bespoke: Using LLM Agents to Generate Just-in-Time Interfaces by Reasoning about User Intent. *ACM*. doi:10.1145/3686215.3688372

[21] Ross, S., Martinez, F., Houde, S., et al. (2023). The Programmer's Assistant: Conversational Interaction with an LLM for Software Development. *ACM IUI*. doi:10.1145/3581641.3584037

[22] Liu, Z., Thompson, J., Wilson, A., et al. (2018). Data Illustrator. *ACM CHI*. doi:10.1145/3173574.3173697

[23] Zong, J., Barnwal, D., Neogy, R., et al. (2020). Lyra 2: Designing Interactive Visualizations by Demonstration. *IEEE TVCG*. doi:10.1109/tvcg.2020.3030367

[24] Tsandilas, T. (2020). StructGraphics: Flexible Visualization Design through Data-Agnostic and Reusable Graphical Structures. *IEEE TVCG*. doi:10.1109/tvcg.2020.3030476

[25] Park, J., An, M., Yang, H., et al. (2026). DirectVis: Editing Code-Based Interactive Visualization with Direct Manipulation. *IEEE PacificVis*. doi:10.1109/pacificvis68791.2026.00014

[26] Shahriari, R., Ragan, E. D., Ruiz, J. (2025). Natural Language Interaction for Editing Visual Knowledge Graphs. *ACM K-CAP*. doi:10.1145/3731443.3771344

[27] Conlen, M., Heer, J. (2018). Idyll: A Markup Language for Authoring and Publishing Interactive Articles on the Web. *ACM UIST*. doi:10.1145/3242587.3242600

[28] Hoque, M. N., Mashiat, T., Ghai, B., et al. (2024). The HaLLMark Effect: Supporting Provenance and Transparent Use of LLMs in Writing with Interactive Visualization. *ACM CHI*. doi:10.1145/3613904.3641895

[29] Gao, T., Yen, H., Yu, J., et al. (2023). Enabling Large Language Models to Generate Text with Citations. *EMNLP*. doi:10.48550/arXiv.2305.14627

[30] Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *EMNLP*. doi:10.48550/arXiv.2305.14251

[31] Hullman, J. R. (2019). Why Authors Don't Visualize Uncertainty. *IEEE TVCG*. doi:10.1109/tvcg.2019.2934287

[32] Zhang, Q., Huang, Y., Lee, J., et al. (2026). Beyond "Made with AI": Visualizing Provenance Density to Mitigate the Transparency Penalty. *IJCAI*. doi:10.24963/ijcai.2026/857

[33] Ghajargar, M., Bardzell, J., Smith-Renner, A., et al. (2021). From "Explainable AI" to "Graspable AI". doi:10.1145/3430524.3442704

[34] Chromik, M., Eiband, M., Buchner, F., et al. (2021). I Think I Get Your Point, AI! The Illusion of Explanatory Depth in XAI. *ACM IUI*. doi:10.1145/3397481.3450644

[35] Feng, Y., Wang, X., Pan, B., et al. (2023). XNLI: Explaining and Diagnosing NLI-Based Visual Data Analysis. *IEEE TVCG*. doi:10.1109/tvcg.2023.3240003

[36] Yang, L., Morizet-Mahoudeaux, P., Guénand, A., et al. (2018). A Design Framework for Instrumenting Analytic Provenance for Problem-Solving Tasks. *Advances in Intelligent Systems and Computing*. doi:10.1007/978-981-10-8612-0_66

[37] Lu, Y., Zheng, M., Crichton, W., et al. (2025). When Do Reactive Notebooks Fail to React? *arXiv:2511.21994*.

[38] DeLine, R. A. (2021). Glinda: Supporting Data Science with Live Programming, GUIs and a DSL. *ACM CHI*. doi:10.1145/3411764.3445267

[39] Zhong, H., Liu, A. (2024). Meerkat: A Distributed Reactive Programming Language with Live Updates. *arXiv:2407.06885*.

[40] Gyarmati, P. F., Manz, T., Moritz, D., et al. (2026). Point, Revise, Review: Grounded Agentic Analysis in Reactive Notebooks with marimo-lens. *arXiv:2609.19839*.

[41] Nikolić, M., Olteanu, D. (2018). Incremental View Maintenance with Triple Lock Factorization Benefits. *ACM SIGMOD*. doi:10.1145/3183713.3183758

[42] Svingos, C., Hernich, A., Gildhoff, H., et al. (2023). Foreign Keys Open the Door for Faster Incremental View Maintenance. *PACMMOD*. doi:10.1145/3588720